Guía para Desarrolladores de Kafka

Integración con Microservicios

Tabla de Contenidos

1. Conceptos Básicos de Kafka
2. Conectarse a Kafka
3. Trabajar con Topics
4. Producir Mensajes (Python)
5. Consumir Mensajes (Python)
6. Grupos de Consumidores
7. Consideraciones con Docker
8. Conexión desde Otros Hosts
9. Buenas Prácticas
10. Operaciones Comunes
11. Solución de Problemas

1. Conceptos Básicos de Kafka

Terminología Clave

- **Topic**: Categoría o canal donde se almacenan los mensajes
- **Partition**: División de un topic para permitir paralelismo
- **Producer**: Aplicación que publica mensajes en topics
- **Consumer**: Aplicación que se suscribe y procesa mensajes
- **Consumer Group**: Grupo de consumidores que procesan mensajes en conjunto

- **Broker**: Servidor Kafka (nuestra instancia corre en localhost:9092)

Nuestra Configuración Actualizada

- **Múltiples listeners**: Soportamos conexiones internas y externas
- **Puerto interno**: 9092 (para conexiones dentro del mismo host/Docker)
- **Puerto externo**: 29092 (para conexiones desde otros servidores)
- **Modo KRaft**: Sin dependencia de ZooKeeper

2. Conectarse a Kafka

Cadenas de Conexión por Entorno

Entorno	Cadena de Conexión	Puerto
Desde tu máquina (localhost)	localhost:9092	9092
Desde otro contenedor Docker	kafka:9092	9092
Desde otro servidor en la red	IP-DEL-SERVIDOR:29092	29092
Desde Kubernetes (mismo cluster)	kafka-service:9092	9092

Verificar Conexión Local

```
# Listar topics para verificar conexión local
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --list
```

3. Trabajar con Topics

Crear un Topic

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic ordenes \
  --partitions 3 \
  --replication-factor 1
```

Listar Topics (local o remoto)

```
# Local
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 --list

# Remoto (desde otro servidor)
kafka-topics.sh --bootstrap-server 192.168.1.100:29092 –list
```

Describir un Topic

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic ordenes
```

Eliminar un Topic

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --delete \
```

```
  --topic ordenes-viejas
```

Modificar un Topic (agregar particiones)

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --alter \
  --topic ordenes \
  --partitions 5
```

4. Producir Mensajes con Python

Instalación de la Librería

```
pip install kafka-python
```

Productor con Soporte Multi-Entorno

```
from kafka import KafkaProducer
import json
import logging
import os

# Configurar logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class OrderProducer:
    def __init__(self, bootstrap_servers=None):
        """
        Inicializa el productor de Kafka con soporte multi-entorno
```

```
    Args:
        bootstrap_servers: Servidor(es) de Kafka. Si es None, usa variable de entorno
    """
    if bootstrap_servers is None:
        # Leer de variable de entorno o usar valor por defecto
        bootstrap_servers = os.getenv('KAFKA_BOOTSTRAP_SERVERS', 'localhost:9092')

    logger.info(f"Conectando a Kafka en: {bootstrap_servers}")

    self.producer = KafkaProducer(
        bootstrap_servers=bootstrap_servers,
        value_serializer=lambda v: json.dumps(v).encode('utf-8'),
        key_serializer=lambda v: v.encode('utf-8') if v else None,
        acks='all',  # Esperar confirmación de todas las réplicas
        retries=5,    # Reintentar en caso de error
        batch_size=16384,  # Tamaño de lote en bytes
        linger_ms=10,  # Esperar 10ms para llenar el lote
        compression_type='gzip',  # Comprimir mensajes
        max_in_flight_requests_per_connection=5  # Máximo mensajes en vuelo
    )

def send_order(self, topic, order_data):
    """
    Envía una orden al topic especificado
```

```
    Args:
        topic: Nombre del topic
        order_data: Diccionario con los datos de la orden
    """
    try:
        # Obtener key del pedido (opcional, para mantener orden por clave)
        order_key = order_data.get('order_id')

        # Enviar mensaje
        future = self.producer.send(
            topic,
            key=order_key,
            value=order_data
        )

        # Esperar confirmación (bloqueante)
        record_metadata = future.get(timeout=10)

        logger.info(f"✅ Mensaje enviado exitosamente:")
        logger.info(f"   Topic: {record_metadata.topic}")
        logger.info(f"   Partición: {record_metadata.partition}")
        logger.info(f"   Offset: {record_metadata.offset}")
        logger.info(f"   Servidor: {self.producer.config['bootstrap_servers']}")
```

```
        return record_metadata

    except Exception as e:
        logger.error(f"✕ Error enviando mensaje: {e}")
        raise

def send_batch(self, topic, orders):
    """
    Envía múltiples órdenes en lote

    Args:
        topic: Nombre del topic
        orders: Lista de diccionarios con datos de órdenes
    """
    for order in orders:
        self.send_order(topic, order)

    # Forzar envío de mensajes pendientes
    self.producer.flush()

def close(self):
    """Cierra el productor"""
    self.producer.flush()
    self.producer.close()
```

```
# Ejemplo de uso con diferentes entornos
if __name__ == "__main__":
    # Datos de ejemplo
    order = {
        'order_id': 'ORD-123456',
        'customer_id': 'CUST-789',
        'amount': 1500.00,
        'items': ['producto1', 'producto2'],
        'timestamp': '2026-03-15T10:30:00'
    }

    # Caso 1: Conexión local (mismo servidor)
    print("=== Conexión Local ===")
    producer_local = OrderProducer('localhost:9092')
    try:
        producer_local.send_order('ordenes', order)
    finally:
        producer_local.close()

    # Caso 2: Conexión remota (otro servidor)
    print("\n=== Conexión Remota ===")
    producer_remote = OrderProducer('192.168.1.100:29092')  # IP del servidor Kafka
    try:
        producer_remote.send_order('ordenes', order)
    finally:
```

```
    producer_remote.close()


# Caso 3: Usando variable de entorno (recomendado para producción)
print("\n=== Usando Variable de Entorno ===")
# Exportar: export KAFKA_BOOTSTRAP_SERVERS="192.168.1.100:29092"
producer_env = OrderProducer()
try:
    producer_env.send_order('ordenes', order)
finally:
    producer_env.close()
```

Productor con Detección Automática de Entorno

```
class SmartOrderProducer:
    def __init__(self):
        """
        Productor que detecta automáticamente si está en Docker o no
        """
        # Detectar si estamos corriendo dentro de Docker
        in_docker = os.path.exists('/.dockerenv')


        if in_docker:
            # Dentro de Docker, usar nombre del servicio
            bootstrap_servers = 'kafka:9092'
        else:
            # Fuera de Docker, usar localhost o IP configurada
            bootstrap_servers = os.getenv('KAFKA_EXTERNAL_SERVERS', 'localhost:29092')
```

```
    logger.info(f"Modo Docker: {in_docker}, Conectando a: {bootstrap_servers}")

    self.producer = KafkaProducer(
        bootstrap_servers=bootstrap_servers,
        value_serializer=lambda v: json.dumps(v).encode('utf-8'),
        key_serializer=lambda v: v.encode('utf-8') if v else None
    )
```

6. Grupos de Consumidores

Ver Grupos de Consumidores

```
# Local
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 --list

# Remoto
kafka-consumer-groups.sh --bootstrap-server 192.168.1.100:29092 –list
```

Describir un Grupo (ver lag)

```
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --group procesamiento-ordenes-remoto
```

Salida:

```
GROUP                    TOPIC     PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
procesamiento-ordenes-remoto ordenes  0          15              20              5
```

```
procesamiento-ordenes-remoto ordenes  1          12             12             0
procesamiento-ordenes-remoto ordenes  2          8              10             2
```

Resetear Offset de un Grupo

```
# Resetear al principio
kafka-consumer-groups.sh --bootstrap-server 192.168.1.100:29092 \
  --group procesamiento-ordenes-remoto \
  --topic ordenes \
  --reset-offsets \
  --to-earliest \
  --execute

# Resetear a fecha específica
kafka-consumer-groups.sh --bootstrap-server 192.168.1.100:29092 \
  --group procesamiento-ordenes-remoto \
  --topic ordenes \
  --reset-offsets \
  --to-datetime 2026-03-15T10:00:00.000 \
  --execute
```

Consideraciones con Docker

docker-compose.yml ACTUALIZADO (con soporte externo)

```
version: '3.8'
services:
  kafka:
```

```
image: apache/kafka:latest
container_name: kafka
ports:
  - "9092:9092"    # Puerto para conexiones internas/localhost
  - "29092:29092"   # Puerto para conexiones externas (otros hosts)
environment:
  KAFKA_NODE_ID: 1
  KAFKA_PROCESS_ROLES: broker,controller
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER

  # Listeners para diferentes tipos de conexión
  KAFKA_LISTENERS: |
    PLAINTEXT://0.0.0.0:9092,
    CONTROLLER://0.0.0.0:9093,
    EXTERNAL://0.0.0.0:29092

  # Publicidad de listeners (CRÍTICO para conexiones externas)
  # Reemplaza 192.168.1.100 con la IP REAL de tu servidor
  KAFKA_ADVERTISED_LISTENERS: |
    PLAINTEXT://kafka:9092,
    EXTERNAL://192.168.1.100:29092

  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: |
    CONTROLLER:PLAINTEXT,
```

```
      PLAINTEXT:PLAINTEXT,
      EXTERNAL:PLAINTEXT

    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    KAFKA_CLUSTER_ID: "5L6g3nShT-eMCtK--X86sw"
  volumes:
    - kafka_data:/var/lib/kafka/data
  networks:
    - kafka-network
  restart: unless-stopped

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: kafka-ui
  ports:
    - "8080:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092  # Usa listener interno
  depends_on:
    - kafka
  networks:
```

```
      - kafka-network
    restart: unless-stopped

networks:
  kafka-network:
    driver: bridge

volumes:
  kafka_data:
    driver: local
```

Configuración por Entorno para Microservicios

Dockerfile del microservicio:

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

# Usar variable de entorno para configuración de Kafka
ENV KAFKA_BOOTSTRAP_SERVERS="kafka:9092"

CMD ["python", "app.py"]
```

docker-compose para microservicio:

```
version: '3.8'

services:

  tu-microservicio:

    build: .

    container_name: mi-app

    depends_on:

      - kafka

    environment:

      # Dentro de Docker, usar nombre del servicio

      KAFKA_BOOTSTRAP_SERVERS: kafka:9092

    networks:

      - kafka-network

    # ... resto de configuración


networks:

  kafka-network:

    external: true  # Usar la red creada por Kafka
```

8. Conexión desde Otros Hosts

Configuración Necesaria en el Servidor Kafka

Para que otros servidores puedan conectarse a tu Kafka, necesitas:

1. **IP Pública/Accesible**: El servidor Kafka debe tener una IP que los clientes puedan alcanzar
2. **Puerto abierto**: El puerto externo (29092) debe estar abierto en el firewall
3. **Listener externo configurado**: Ya lo incluimos en el docker-compose.yml

Verificar Configuración desde el Servidor Kafka

```
# Verificar que Kafka está escuchando en el puerto externo
sudo netstat -tlnp | grep 29092

# Verificar configuración de listeners
docker exec kafka cat /etc/kafka/kraft/server.properties | grep -i listener

# Probar conexión local al puerto externo
nc -zv localhost 29092
```

Configurar Firewall (ejemplo con ufw)

```
# Abrir puerto externo
sudo ufw allow 29092/tcp

# Verificar reglas
sudo ufw status
```

Probar Conexión desde el Cliente Remoto

```
# Instalar herramientas de Kafka (si no están)
wget https://downloads.apache.org/kafka/3.5.0/kafka_2.13-3.5.0.tgz
tar -xzf kafka_2.13-3.5.0.tgz
cd kafka_2.13-3.5.0

# Probar conexión
bin/kafka-topics.sh --bootstrap-server 192.168.1.100:29092 –list
```

Script Python para probar conexión remota:

```
# test_remote_connection.py
```

```
from kafka import KafkaProducer, KafkaConsumer
from kafka.errors import NoBrokersAvailable
import socket
import sys

def test_connection(host, port):
    """Prueba conectividad TCP básica"""
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(5)
        result = sock.connect_ex((host, port))
        if result == 0:
            print(f"✅ Puerto {port} abierto en {host}")
        else:
            print(f"❌ No se puede conectar a {host}:{port} (código: {result})")
        sock.close()
        return result == 0
    except Exception as e:
        print(f"❌ Error: {e}")
        return False

def test_kafka_connection(bootstrap_servers):
    """Prueba conexión con Kafka"""
    try:
        producer = KafkaProducer(
```

```
            bootstrap_servers=bootstrap_servers,
            max_block_ms=5000
        )
        producer.close()
        print(f"✅ Conexión Kafka exitosa a {bootstrap_servers}")
        return True
    except NoBrokersAvailable:
        print(f"❌ No se puede conectar a Kafka en {bootstrap_servers}")
        return False
    except Exception as e:
        print(f"❌ Error: {e}")
        return False

if __name__ == "__main__":
    KAFKA_HOST = "192.168.1.100"  # IP de tu servidor Kafka
    KAFKA_PORT = 29092

    print(f"Probando conexión a Kafka en {KAFKA_HOST}:{KAFKA_PORT}")
    print("-" * 50)

    # Probar conectividad TCP
    if test_connection(KAFKA_HOST, KAFKA_PORT):
        # Probar conexión Kafka
        test_kafka_connection(f"{KAFKA_HOST}:{KAFKA_PORT}")
```

Ejemplo de Configuración para Cliente Remoto

config.py para aplicación:

```
import os

import socket


class KafkaConfig:

    @staticmethod

    def get_bootstrap_servers():

        """

        Determina los servidores de Kafka según el entorno

        """

        # Variable de entorno tiene prioridad

        env_servers = os.getenv('KAFKA_BOOTSTRAP_SERVERS')

        if env_servers:

            return env_servers


        # Detectar si estamos en el mismo servidor que Kafka

        try:

            # Intentar conexión local primero

            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

            sock.settimeout(1)

            result = sock.connect_ex(('localhost', 9092))

            sock.close()


            if result == 0:
```

```
        return 'localhost:9092'  # Mismo servidor
    except:
        pass

    # Por defecto, usar IP del servidor Kafka remoto
    # En producción, esto vendría de un service discovery / configuración
    return os.getenv('KAFKA_REMOTE_SERVERS', '192.168.1.100:29092')

# Uso en tu código
bootstrap_servers = KafkaConfig.get_bootstrap_servers()
producer = KafkaProducer(bootstrap_servers=bootstrap_servers)
```

9. Buenas Prácticas

1. Diseño de Topics

- **Nombres descriptivos:** ordenes-creadas, pagos-procesados, inventario-actualizado
- **Particiones suficientes:** Al menos el número de consumidores esperados × 2
- **Retención:** Configurar según necesidades (7 días por defecto)

2. Productor - Mejores Prácticas

```
# Configuración recomendada para producción
producer = KafkaProducer(
    bootstrap_servers=servers,
    acks='all',             # Esperar confirmación de todas las réplicas
    retries=5,              # Reintentar en errores transitorios
    batch_size=32768,       # 32KB batch
```

```
    linger_ms=20,            # Esperar 20ms para llenar batch
    compression_type='snappy', # Comprimir mensajes
    max_in_flight_requests_per_connection=5  # Máximo 5 mensajes en vuelo
)
```

3. Consumidor - Mejores Prácticas

```
# Procesamiento idempotente (CRÍTICO para conexiones remotas)
def process_message(message):
    order_id = message['order_id']

    # Verificar si ya procesamos este mensaje
    # (las redes pueden causar duplicados)
    if redis_client.sismember('processed_orders', order_id):
        logger.info(f"Orden {order_id} ya procesada, ignorando")
        return

    # Procesar orden
    save_to_database(message)

    # Marcar como procesada
    redis_client.sadd('processed_orders', order_id)
    redis_client.expire('processed_orders', 86400)  # Expirar después de 24h
```

4. Manejo de Reconexiones para Clientes Remotos

```
class ResilientKafkaClient:
    def __init__(self, bootstrap_servers):
        self.bootstrap_servers = bootstrap_servers
```

```
    self.producer = None
    self.reconnect()

def reconnect(self):
    """Reconecta a Kafka con reintentos"""
    max_retries = 5
    for attempt in range(max_retries):
        try:
            self.producer = KafkaProducer(
                bootstrap_servers=self.bootstrap_servers,
                max_block_ms=10000
            )
            logger.info("Reconexión exitosa")
            return
        except Exception as e:
            wait_time = 2 ** attempt  # Exponential backoff
            logger.warning(f"Intento {attempt + 1} falló, reintentando en {wait_time}s")
            time.sleep(wait_time)

    raise Exception("No se pudo conectar a Kafka después de múltiples intentos")

def send_with_retry(self, topic, value):
    """Envía mensaje con reconexión automática"""
    for attempt in range(3):
        try:
```

```
            if not self.producer:
                self.reconnect()

            future = self.producer.send(topic, value=value)
            return future.get(timeout=10)

        except (KafkaError, ConnectionError) as e:
            logger.error(f"Error enviando mensaje: {e}")
            self.producer = None
            time.sleep(2 ** attempt)

    raise Exception("No se pudo enviar el mensaje después de reintentos")
```

10. Operaciones Comunes

Script de Monitoreo Multi-Entorno

```
cat > check-kafka.sh << 'EOF'
#!/bin/bash

KAFKA_LOCAL="localhost:9092"
KAFKA_REMOTO="192.168.1.100:29092"

echo "📊 VERIFICACIÓN DE KAFKA"
echo "========================"

# Función para verificar conexión
```

```
check_connection() {
    local servers=$1
    local name=$2

    echo -e "\n🔍 Probando conexión a $name ($servers)"

    # Probar conectividad básica
    host=$(echo $servers | cut -d: -f1)
    port=$(echo $servers | cut -d: -f2)

    if nc -zv $host $port 2>/dev/null; then
        echo "✅ Puerto $port accesible"

        # Probar lista de topics
        if kafka-topics.sh --bootstrap-server $servers --list &>/dev/null; then
            echo "✅ Conexión Kafka exitosa"

            echo -e "\n📋 Topics:"
            kafka-topics.sh --bootstrap-server $servers --list

            echo -e "\n👥 Grupos:"
            kafka-consumer-groups.sh --bootstrap-server $servers --list 2>/dev/null
        else
            echo "❌ No se puede conectar a Kafka"
        fi
```

```
    else
        echo "✗ Puerto $port no accesible"
    fi
}

# Verificar conexiones
check_connection $KAFKA_LOCAL "LOCAL"
check_connection $KAFKA_REMOTO "REMOTO"
EOF

chmod +x check-kafka.sh
./check-kafka.sh
```

Ver Mensajes desde Cliente Remoto

```
# remote_peek.py
from kafka import KafkaConsumer
import json
import sys

def peek_remote_topic(host, topic, n_messages=5):
    """
    Inspecciona un topic remoto
    """
    consumer = KafkaConsumer(
        topic,
        bootstrap_servers=f"{host}:29092",
```

```
        auto_offset_reset='earliest',
        enable_auto_commit=False,
        value_deserializer=lambda v: json.loads(v.decode('utf-8')),
        consumer_timeout_ms=10000  # Timeout después de 10s sin mensajes
    )

    messages = []
    try:
        for i, msg in enumerate(consumer):
            if i >= n_messages:
                break
            messages.append({
                'offset': msg.offset,
                'key': msg.key.decode() if msg.key else None,
                'value': msg.value,
                'partition': msg.partition
            })
    except Exception as e:
        print(f"Error: {e}")
    finally:
        consumer.close()

    return messages

if __name__ == "__main__":
```

```
KAFKA_HOST = "192.168.1.100"  # IP de tu servidor Kafka


for msg in peek_remote_topic(KAFKA_HOST, 'ordenes', 5):
    print(f"Offset: {msg['offset']}")
    print(f"Valor: {msg['value']}")
    print("-" * 40)
```

11. Solución de Problemas

Errores Comunes en Conexiones Remotas

Error	Causa	Solución
Connection refused	Puerto no accesible	Verificar firewall: sudo ufw status
NoBrokersAvailable	IP/hostname incorrecto	Verificar IP: ip addr show
TimeoutException	Red lenta o firewall	Aumentar timeouts en cliente
Leader not available	Kafka iniciando	Esperar 10 segundos
UnknownHostException	DNS no resuelve	Usar IP directamente

Diagnóstico Rápido para Conexiones Remotas

En el servidor Kafka:

```
# 1. Verificar que Kafka está corriendo
docker ps | grep kafka
```

2. Verificar puertos expuestos

docker port kafka

3. Verificar listeners configurados

docker exec kafka cat /etc/kafka/kraft/server.properties | grep -A5 listeners

4. Verificar IP del servidor

ip addr show | grep inet

5. Verificar firewall

sudo ufw status

sudo iptables -L -n | grep 29092

En el cliente remoto:

1. Probar conectividad básica

ping 192.168.1.100

2. Probar puerto específico

nc -zv 192.168.1.100 29092

3. Probar con telnet

telnet 192.168.1.100 29092

4. Verificar resolución DNS

nslookup kafka-server # si usas nombre DNS

Script de Diagnóstico Completo

```
# diagnostic.py

import socket

import sys

from kafka import KafkaProducer

from kafka.errors import NoBrokersAvailable


def diagnose_connection(host, port):

    print(f"\n🔍 Diagnosticando conexión a {host}:{port}")

    print("=" * 50)


    # 1. Resolución DNS

    try:

        ip = socket.gethostbyname(host)

        print(f"✅ DNS: {host} -> {ip}")

    except socket.gaierror:

        print(f"❌ DNS: No se puede resolver {host}")

        return


    # 2. Ping básico

    response = os.system(f"ping -c 1 -W 2 {host} > /dev/null 2>&1")

    if response == 0:

        print("✅ Ping: Host responde")

    else:

        print("❌ Ping: Host no responde")
```

3. Conexión TCP

sock = socket.socket